

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



ĐỀ TÀI: HƯỚNG TIẾP CẬN MỚI CHO HỆ THỐNG TƯ VẤN MUA BÁN
XE

Học phần: ĐỒ ÁN TỐT NGHIỆP

Giảng viên hướng dẫn: PGS.TS NGUYỄN THANH BÌNH

Thành viên nhóm:

Nguyễn Văn Trung Chính - 22280007

Nguyễn Xuân Việt Đức - 22280012

Bành Đức Khánh - 22280044

Lê Trọng Nghĩa - 22280059

TP. Hồ Chí Minh, Việt Nam

Tháng 7, 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan đồ án tốt nghiệp với đề tài **“Hướng tiếp cận mới cho hệ thống tư vấn mua bán xe”** là công trình nghiên cứu của nhóm chúng em dưới sự hướng dẫn của **PGS.TS Nguyễn Thanh Bình**.

Các số liệu, kết quả trình bày trong đồ án là trung thực và chưa từng được ai công bố trong bất kỳ công trình nào khác. Tất cả các nguồn tài liệu tham khảo đều được trích dẫn đầy đủ và rõ ràng.

Nhóm chúng em xin chịu hoàn toàn trách nhiệm về nội dung của đồ án này.

TP. Hồ Chí Minh, tháng 7, 2026

Nhóm sinh viên thực hiện

Nguyễn Văn Trung Chính

Nguyễn Xuân Việt Đức

Bành Đức Khánh

Lê Trọng Nghĩa

THUYẾT MINH CHỈNH SỬA ĐỀ TÀI

STT	Nội dung góp ý	Nội dung chỉnh sửa	Trang
1			
2			
3			
4			
5			

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

PGS.TS Nguyễn Thanh Bình

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, nhóm em đã nhận được sự hướng dẫn, hỗ trợ và tạo điều kiện từ quý thầy cô hướng dẫn và nhà trường. Bằng tất cả sự trân trọng, nhóm em xin gửi lời cảm ơn sâu sắc đến quý thầy cô hướng dẫn và nhà trường đã đồng hành cùng nhóm em trong quá trình thực hiện đề tài.

Lời đầu tiên, em xin bày tỏ lòng biết ơn đến thầy PGS.TS Nguyễn Thanh Bình, giảng viên hướng dẫn, người đã nhiệt tình định hướng, góp ý và hỗ trợ nhóm em trong suốt thời gian nghiên cứu và hoàn thiện khóa luận. Những nhận xét và sự hỗ trợ tận tâm của thầy là nền tảng quan trọng giúp chúng em hoàn thành tốt đề tài.

Chúng em cũng xin gửi lời cảm ơn đến Khoa Toán - Tin – Trường Đại học Khoa Học Tự Nhiên TP. Hồ Chí Minh đã tạo môi trường học tập thuận lợi, cung cấp kiến thức chuyên môn và kỹ năng cần thiết trong suốt quá trình học tập tại trường.

Và, em xin cảm ơn các thầy và các thầy/cô bộ môn đã truyền đạt những kiến thức bổ ích và luôn tận tâm trong công tác giảng dạy, giúp chúng em có được nền tảng vững chắc để thực hiện đề tài này.

Chúng em đã cố gắng hoàn thành đề tài, song kết quả cuối cùng có thể khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý của quý thầy cô để hoàn thiện hơn trong tương lai.

TP. Hồ Chí Minh, tháng 7, 2026

Chúng em trân trọng cảm ơn thầy !

Mục lục

Lời cam đoan	i
Nhận xét của giảng viên hướng dẫn	ii
Nhận xét của giảng viên phản biện	iii
Thuyết minh chỉnh sửa đề tài	iv
Lời cảm ơn	v
1 TỔNG QUAN	1
1.1 Sơ lược về dự án	1
1.2 Kiến trúc hệ thống tổng quát	1
1.3 Giới thiệu về nguồn dữ liệu	2
1.3.1 Tầng Bronze (Dữ liệu thô)	2
1.3.2 Tầng Silver (Dữ liệu chuẩn hóa)	2
1.3.3 Tầng Gold (Dữ liệu phục vụ ứng dụng)	3
1.4 Ý nghĩa và Thách thức	3
2 QUÁ TRÌNH THU THẬP VÀ XỬ LÝ DỮ LIỆU	5
2.1 Xác định thông tin cần lưu trữ	5
2.2 Tiến hành thu thập dữ liệu	5
2.2.1 Vượt qua cơ chế chống bot (Cloudflare Turnstile)	5
2.2.2 Kiến trúc crawler	5
2.2.3 Điều phối tự động bằng Temporal	6
2.3 Xử lý và chuyển đổi dữ liệu (dbt Pipeline)	7
2.3.1 Nạp dữ liệu vào tầng Bronze (load_bronze)	7
2.3.2 Chuyển đổi dữ liệu bằng dbt (Medallion Architecture)	7
2.3.3 Refresh materialized view và tính toán ML	8
2.3.4 Tổng kết luồng xử lý dữ liệu	8
3 BACKEND VÀ HỆ THỐNG ĐỀ XUẤT	10
3.1 Giới thiệu	10
3.2 Kiến trúc tổng quan	11
3.3 Kiến trúc API	12
3.4 Thiết kế Database	13
3.5 Data Pipeline	14
3.6 Hệ thống Authentication	14
3.7 Caching và Tối ưu hiệu năng	14
3.7.1 Caching bằng Redis	14

3.7.2	Tối ưu truy vấn Database	15
3.8	Hệ thống Recommendation	16
3.8.1	Stage 1 — Candidate Generation	17
3.8.2	Stage 2 — Ranking	17
3.8.3	Stage 3 — Re-ranking	18
3.8.4	Tóm tắt luồng xử lý	18
4	XÂY DỰNG CHATBOT	19
4.1	Mục tiêu và phạm vi tư vấn	19
4.2	Mô hình cơ sở xây dựng chatbot – Agentic RAG	19
4.3	Xây dựng Vector Database cho ChatBot	20
4.4	Kiến trúc Agentic: LangGraph StateGraph	21
4.4.1	Hồ sơ người dùng (User Profile Slot-Fill)	21
4.4.2	Phân loại ý định (Intent Routing)	21
4.5	Hệ thống Hybrid Retrieval và Sinh câu trả lời	21
4.5.1	Hybrid Retrieval	21
4.5.2	Các nhánh xử lý chuyên biệt	22
4.5.3	Sinh câu trả lời (Generation)	22
4.5.4	Chuẩn hóa câu hỏi (Standalone Question)	22
4.6	Hạn chế và định hướng phát triển	22
4.6.1	Hạn chế hiện tại	22
4.6.2	Định hướng phát triển	22
5	TỔNG KẾT	24
5.1	Kết Luận	24
5.2	Ứng Dụng Thực Tế	24

1 TỔNG QUAN

1.1 Sơ lược về dự án

Trong bối cảnh kỷ nguyên số và xu hướng chuyển đổi số diễn ra mạnh mẽ, mua sắm qua các nền tảng trực tuyến đã trở thành thói quen phổ biến của người dùng tại Việt Nam cũng như trên toàn thế giới. Cùng với đó, thị trường mua bán ô tô đang phát triển sôi động, với số lượng mẫu xe đa dạng và mức giá biến động theo nhiều yếu tố như thương hiệu, phiên bản, tình trạng xe và tính năng đi kèm. Sự phong phú về lựa chọn giúp người mua có nhiều cơ hội tiếp cận sản phẩm phù hợp, nhưng cũng làm tăng khó khăn trong việc so sánh, đánh giá và xác định phương án tối ưu theo nhu cầu và ngân sách.

Xuất phát từ thực tế đó, nhóm đề xuất phát triển một hệ thống tư vấn và gợi ý xe nằm trên hạ tầng đám mây với tính sẵn sàng cao dựa trên dữ liệu thu thập từ website bán xe **Cars.com**, bao gồm nhiều mức giá và tình trạng xe khác nhau. Hệ thống tập trung vào việc tổng hợp thông tin, chuẩn hóa dữ liệu và hỗ trợ người dùng sàng lọc theo các tiêu chí quan trọng như khoảng giá, dòng xe, mục đích sử dụng và một số thông số kỹ thuật cơ bản. Từ đó, hệ thống hiển thị danh sách lựa chọn phù hợp hơn, giúp người dùng nhanh chóng khoanh vùng các mẫu xe đáp ứng đúng nhu cầu.

Với bộ dữ liệu hơn **6.000 xe** (VIN duy nhất) đã thu thập và cập nhật hàng tuần, dự án không chỉ dừng lại ở việc gợi ý lựa chọn hiện tại mà còn hướng tới việc mở rộng trong việc phân tích xu hướng thị trường trong tương lai. Cụ thể, từ biến động giá theo thời gian và mức độ quan tâm của người mua, hệ thống có thể hỗ trợ nhận định xu hướng tăng hoặc giảm giá và tình hình cung cầu của một số dòng xe. Bên cạnh đó, việc thu thập và khai thác các nhận xét của khách hàng cũng tạo cơ hội cho việc phân tích cảm xúc và nhu cầu người dùng đối với từng dòng xe, qua đó bổ sung thêm một lớp thông tin hữu ích cho quá trình ra quyết định.

Về giải pháp kỹ thuật, dự án phát triển hệ thống theo ba hướng chính:

- **Recommender System (Hệ thống đề xuất):** Hệ thống đề xuất lai (hybrid) 3 giai đoạn bao gồm: (1) sinh ứng viên từ 4 nguồn (Collaborative Filtering qua bảng item_similarity, Content-based, Vector search qua Qdrant, Popularity), (2) xếp hạng bằng bộ chấm điểm tuyến tính 7 đặc trưng, (3) tái xếp hạng bằng MMR để đảm bảo đa dạng kết quả.
- **Agentic Chatbot (Chatbot tư vấn thông minh):** Xây dựng chatbot dạng agent sử dụng **Lang-Graph** với luồng hội thoại đa nhánh (7 loại intent: compare, analytics, specs, specific, vague, chitchat, off_topic). Chatbot kết hợp truy vấn SQL trực tiếp trên gold.vehicles và tìm kiếm ngữ nghĩa qua Qdrant để đưa ra tư vấn cá nhân hóa, có dẫn chứng dữ liệu thực.
- **Data Pipeline tự động:** Hệ thống pipeline dữ liệu tự động chạy hàng tuần sử dụng **Temporal** để điều phối toàn bộ luồng: thu thập dữ liệu (crawl) → chuyển đổi dữ liệu (dbt medallion) → tính toán ML (cosine similarity, embeddings).



Hình 1.1: *Cars.com* – Trang web chính được sử dụng để thu thập dữ liệu.

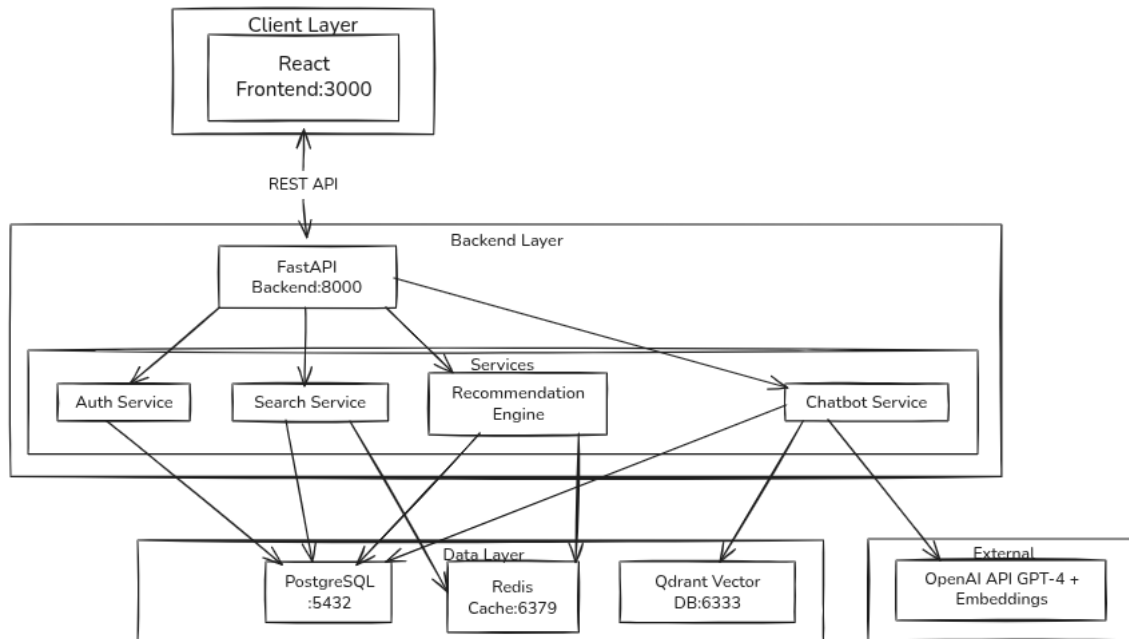
1.2 Kiến trúc hệ thống tổng quát

Hệ thống được xây dựng trên nền tảng đám mây Google Cloud Platform (GCP) với kiến trúc microservices, bao gồm các thành phần chính như sau:

- **Frontend:** Ứng dụng web đơn trang (SPA) xây dựng bằng React + TypeScript + Tailwind CSS, triển

khởi trên Google Cloud Run dưới dạng container nginx.

- **Backend:** API server xây dựng bằng FastAPI (Python), triển khai trên Google Cloud Run, cung cấp các endpoint cho tìm kiếm, đề xuất, chat, xác thực và quản lý tương tác người dùng.
- **Database:** AlloyDB (PostgreSQL 17) lưu trữ dữ liệu theo kiến trúc medallion (bronze/silver/gold). Qdrant Cloud làm vector database cho tìm kiếm ngữ nghĩa. Redis làm bộ nhớ đệm.
- **Data Pipeline:** Temporal server tự triển khai (self-hosted) trên GCE VM điều phối toàn bộ luồng thu thập và xử lý dữ liệu tự động hàng tuần.
- **Chuyển đổi dữ liệu:** dbt (data build tool) thực hiện chuyển đổi dữ liệu từ bronze thô sang silver (chuẩn hóa, 3NF) và gold (bảng phục vụ ứng dụng).



Hình 1.2: Kiến trúc tổng quát của hệ thống tư vấn mua bán xe.

1.3 Giới thiệu về nguồn dữ liệu

Bộ dữ liệu được tổ chức theo **kiến trúc medallion** (Bronze → Silver → Gold) trên cơ sở dữ liệu PostgreSQL (AlloyDB), với dữ liệu được chuyển đổi tự động bằng dbt.

1.3.1 Tầng Bronze (Dữ liệu thô)

Sau khi thu thập dữ liệu trên trang web Cars.com, dữ liệu thô được lưu trữ trên **Google Cloud Storage (GCS)** theo cấu trúc phân vùng ngày:

- Mỗi file JSON chứa toàn bộ thông tin của một chiếc xe (thông tin bài đăng, thông số kỹ thuật, tính năng, hình ảnh, thông tin người bán, đánh giá).
- Cấu trúc lưu trữ: `gs://incremental_raw/dt=YYYY-MM-DD/<page>/<vin>.json`
- Hình ảnh xe và người bán được lưu kèm theo trong thư mục con.

Dữ liệu từ GCS sau đó được nạp vào bảng `bronze.raw_listings` trên PostgreSQL dưới dạng JSONB. Bảng này là **append-only** — mỗi file JSON tương ứng một hàng, khóa idempotent là `file_hash` (SHA-256 của nội dung file). Việc chạy lại pipeline không tạo bản ghi trùng lặp.

1.3.2 Tầng Silver (Dữ liệu chuẩn hóa)

Tầng Silver do **dbt** tự động xây dựng từ tầng Bronze, bao gồm hai bước:

1. **Staging:** Giải trùng lặp theo VIN (lấy bản ghi mới nhất theo `crawl_date`), sau đó trích xuất và phẳng

hóa (flatten) cấu trúc JSON lồng nhau thành các trường riêng biệt.

2. **Mô hình chiều (Dimensional model):** Tổ chức thành bảng fact và dimension theo mô hình 3NF:

- **fct_listing:** Bảng fact chính — mỗi hàng là một bài đăng xe (giá, quãng đường, tình trạng, động cơ, v.v.)
- **dim_car_model:** Thông tin hãng xe và dòng xe
- **dim_seller:** Thông tin người bán/đại lý
- **dim_feature:** Danh mục tính năng xe
- **dim_listing_image:** Hình ảnh xe
- **bridge_listing_feature:** Liên kết N-N giữa bài đăng và tính năng
- **fct_model_rating** và **fct_model_review:** Đánh giá và nhận xét theo dòng xe

1.3.3 Tầng Gold (Dữ liệu phục vụ ứng dụng)

Tầng Gold cũng do dbt xây dựng, là các bảng denormalized tối ưu cho truy vấn từ backend. Các bảng chính:

- **gold.vehicles:** Bảng chính (merge by VIN, incremental) — một hàng mỗi xe, chứa tất cả thông tin cần thiết (giá, hãng, dòng, tính năng, hình ảnh chính, đánh giá mô hình). Hiện tại có **6.072 VIN** duy nhất.
- **gold.vehicle_price_history:** Lịch sử giá xe theo ngày thu thập.
- **gold.car_models:** Thông tin dòng xe tổng hợp (đánh giá, tỷ lệ khuyến nghị).
- **gold.sellers:** Thông tin đại lý/người bán.
- **gold.reviews:** Nhận xét chi tiết từ người dùng.
- **gold.vehicle_features** và **gold.vehicle_images:** Tính năng và hình ảnh xe.

Ngoài ra, tầng Gold còn bao gồm các **bảng user-domain** do ứng dụng FastAPI ghi trực tiếp (không qua dbt):

- **gold.users:** Thông tin người dùng (đăng ký, xác thực).
- **gold.user_interactions:** Lịch sử tương tác (view, click, compare, save, favorite, contact, inquiry).
- **gold.item_similarity:** Ma trận tương đồng xe (tính bằng cosine similarity, cập nhật hàng tuần).
- **gold.chat_sessions** và **gold.chat_messages:** Lịch sử hội thoại chatbot của người dùng đã đăng nhập.

1.4 Ý nghĩa và Thách thức

Bộ dữ liệu thu thập từ *Cars.com* mang lại một số **ưu điểm** đáng chú ý:

- **Tính cập nhật cao:** Dữ liệu được thu thập tự động hàng tuần qua pipeline Temporal, đảm bảo thông tin bài đăng và giá bán phản ánh tương đối sát thị trường tại thời điểm hiện tại.
- **Thông tin đa dạng:** Dữ liệu bao gồm nhiều khía cạnh như thông tin cơ bản của xe, các tính năng (feature), hình ảnh, thông tin người bán và đánh giá từ người dùng.
- **Nguồn dữ liệu phổ biến:** Website có lượng người dùng lớn, giúp thu thập được nhiều nhận xét thực tế, hỗ trợ phân tích hành vi và nhu cầu của người mua.
- **Kiến trúc dữ liệu chuẩn:** Áp dụng kiến trúc medallion (Bronze → Silver → Gold) với dbt, đảm bảo dữ liệu được chuẩn hóa, có thể truy vết nguồn gốc (lineage) và dễ mở rộng.
- **Idempotent và incremental:** Pipeline thu thập và xử lý dữ liệu đảm bảo tính idempotent — chạy lại bất kỳ bước nào cũng không tạo dữ liệu trùng lặp, nhờ cơ chế dedup theo **file_hash** (bronze) và merge theo VIN (gold).

Tuy nhiên, bộ dữ liệu cũng tồn tại một số **thách thức**:

- **Chống bot (Anti-scraping):** Cars.com sử dụng hệ thống bảo vệ Cloudflare Turnstile, khiến việc thu thập dữ liệu bằng trình duyệt headless thông thường thất bại. Nhóm phải sử dụng SeleniumBase UC mode (Undetected ChromeDriver) kết hợp Xvfb trên máy thật để vượt qua cơ chế này. Hệ quả là

crawler **không thể chạy trong Docker container** mà phải chạy trực tiếp trên máy host.

- **Về mặt địa lý:** Dữ liệu chủ yếu phản ánh thị trường ô tô tại Mỹ, do đó giá cả có thể lệch so với các quốc gia khác một số thời điểm.
- **Về mặt thời gian:** Giá và tình trạng xe thay đổi nhanh theo thời điểm. Mặc dù hệ thống đã có cơ chế thu thập hàng tuần, nhưng vẫn tồn tại độ trễ nhất định.
- **Về chất lượng dữ liệu:** Một số bài đăng có thể thiếu thông tin hoặc trình bày không đầy đủ; đồng thời người bán có thể không công bố (hoặc mô tả mơ hồ) các khuyết điểm của xe, gây khó khăn trong việc đối soát và đánh giá chính xác tình trạng xe thực tế.

Nhìn chung, bộ dữ liệu này là nền tảng quan trọng để phát triển các chức năng sàng lọc, gợi ý xe theo nhu cầu và xây dựng chatbot tư vấn dựa trên hệ thống đề xuất lai (hybrid Recommender System) và chatbot thông minh dạng agent (Agentic LangGraph chatbot).

2 QUÁ TRÌNH THU THẬP VÀ XỬ LÝ DỮ LIỆU

2.1 Xác định thông tin cần lưu trữ

Nhóm tiến hành khảo sát website *Cars.com* nhằm xác định cấu trúc hiển thị, phân loại nội dung và các trường thông tin có thể trích xuất. Kết quả khảo sát cho thấy dữ liệu trên website có cấu trúc tương đối thống nhất giữa các trang bài đăng, tạo điều kiện thuận lợi để tự động hóa quá trình thu thập và chuẩn hóa dữ liệu.

Nhằm mở rộng phạm vi dữ liệu, không chỉ giới hạn ở các thông số và nội dung bài đăng xe, nhóm tiếp tục thu thập thêm bằng cách điều hướng sang trang người bán (seller/dealer) và trang đánh giá xe (review). Cách tiếp cận này giúp bổ sung thông tin về đại lý, mức độ uy tín, cũng như các nhận xét thực tế từ người dùng. Dữ liệu sau đó được phân thành các nhóm chính như sau:

- **Thông tin bài đăng xe (Post):** trạng thái xe (mới hoặc cũ), tiêu đề, giá bán, số km đã đi, phương thức thanh toán trả góp, hình ảnh, và đường dẫn đến các trang chi tiết.
- **Thông tin chi tiết kỹ thuật (Vehicle Details):** Các thuộc tính cơ bản như hãng xe, dòng xe, năm sản xuất, loại động cơ, hộp số, nhiên liệu, tình trạng bảo hành, và mã VIN. Đây là nhóm thông tin quan trọng nhất, giúp định danh và phân loại xe.
- **Tính năng xe (Features):** Bao gồm danh sách các tính năng được chia theo nhóm như an toàn (Safety), tiện nghi (Convenience), công nghệ (Technology), hỗ trợ lái (Driver Assist), v.v.
- **Người bán (Seller/Dealer):** Thông tin đại lý hoặc cá nhân đăng bán gồm tên, địa chỉ, liên hệ, thời gian mở cửa, điểm đánh giá và mô tả.
- **Nhận xét của người dùng (Reviews/Ratings):** Thông tin phản hồi từ khách hàng như điểm đánh giá tổng quan, nội dung nhận xét, thời gian đánh giá, vị trí, và điểm chi tiết theo từng tiêu chí (Performance, Comfort, Interior, Reliability, Value, Exterior).

Sau khi khảo sát, nhóm nhận thấy mỗi xe sẽ mang một số VIN duy nhất nên đã quyết định lựa chọn mã VIN (Vehicle Identification Number) làm khóa định danh duy nhất cho mỗi bài đăng của xe, dùng để liên kết giữa các bảng dữ liệu trong giai đoạn xử lý và lưu trữ.

2.2 Tiến hành thu thập dữ liệu

2.2.1 Vượt qua cơ chế chống bot (Cloudflare Turnstile)

Thách thức lớn nhất trong quá trình thu thập dữ liệu là hệ thống bảo vệ **Cloudflare Turnstile** mà *Cars.com* sử dụng. Cơ chế này phát hiện và chặn các trình duyệt tự động (headless browser) thông thường, khiến các thư viện crawling phổ biến như Selenium headless hay Scrapy không thể truy cập nội dung trang.

Để giải quyết vấn đề này, nhóm sử dụng **SeleniumBase** ở chế độ **UC mode** (Undetected ChromeDriver) — một phương pháp ngắt kết nối ChromeDriver trong quá trình tải trang để Cloudflare không thể nhận diện (fingerprint) trình duyệt tự động. Khi phát hiện trang challenge “Just a moment...” hoặc “Verifying...”, hệ thống tự động gọi `uc_gui_click_captcha()` để giải captcha Turnstile.

Hệ quả quan trọng: crawler **phải chạy trên máy thật** (host) với trình duyệt Chrome đầy đủ và **Xvfb** (X Virtual Framebuffer) để mô phỏng màn hình ảo. Việc chạy trong Docker container là không khả thi vì Turnstile phát hiện môi trường container.

2.2.2 Kiến trúc crawler

Quá trình thu thập dữ liệu được xây dựng dưới dạng package Python (**crawler/**) với kiến trúc module hóa, chia thành ba giai đoạn chính:

Giai đoạn 1: Thu thập danh sách liên kết (`link_crawler.py`)

Module `link_crawler.py` sử dụng một phiên trình duyệt SeleniumBase duy nhất để duyệt các trang kết quả tìm kiếm trên *Cars.com*. Trình duyệt duy trì cookie Cloudflare xuyên suốt các trang để tránh bị challenge lặp lại. Mỗi trang kết quả chứa khoảng 20–25 liên kết bài đăng dạng `/vehicledetail/...`

Danh sách URL được lưu theo từng trang trong thư mục `car_links/page_{n}.txt`. Cơ chế **resumable**: nếu file liên kết đã tồn tại và không rỗng, trang đó sẽ được bỏ qua. Giữa các trang có khoảng nghỉ ngẫu nhiên (2–3.5 giây) để tránh bị rate limit. Khi gặp lỗi, hệ thống tự động retry với backoff lũy thừa (tối đa 3 lần).

Giai đoạn 2: Thu thập chi tiết từng xe (`detail_scraper.py`)

Từ danh sách URL đã thu được, module `detail_scraper.py` mở nhiều phiên trình duyệt song song (mặc định tối đa 5 worker) để tăng tốc. Mỗi worker xử lý một phần danh sách URL và lưu kết quả ra file JSON riêng biệt.

Đối với mỗi URL xe, hệ thống thực hiện quy trình `scrape_full` gồm các bước:

1. **Scrape trang chi tiết xe (`scrape_listing`):** Tải HTML qua SeleniumBase UC mode, sau đó sử dụng **BeautifulSoup** để phân tích cú pháp (parse) và trích xuất ba nhóm thông tin qua các parser chuyên biệt:
 - **parse_post**: Trích xuất thông tin bài đăng (giá, trạng thái new/used, tiêu đề, quảng đường, thanh toán trả góp, hình ảnh, mã VIN).
 - **parse_seller**: Trích xuất thông tin người bán từ sidebar (tên đại lý, địa chỉ, đánh giá, liên kết dealer page).
 - **parse_car**: Trích xuất thông số kỹ thuật (động cơ, hộp số, nhiên liệu, drivetrain), danh sách tính năng theo nhóm (Safety, Comfort, Technology, ...), và thông tin review tổng hợp (điểm đánh giá, tỷ lệ khuyến nghị, nhận xét chi tiết theo tiêu chí).
2. **Scrape trang đại lý (`scrape_dealer`):** Nếu trang xe có liên kết đến trang dealer, hệ thống truy cập thêm để thu thập số điện thoại, giờ hoạt động, và thông tin bổ sung.
3. **Hợp nhất và lưu JSON:** Toàn bộ dữ liệu được hợp nhất thành một cấu trúc JSON duy nhất cho mỗi xe, lưu tại `raw_data/{page}/{index}.json`.

Cơ chế an toàn: mỗi file đã tồn tại sẽ được bỏ qua (skip), cho phép tiếp tục từ điểm dừng nếu quá trình bị gián đoạn. Mỗi URL thất bại sau 3 lần retry sẽ được ghi log nhưng không dừng pipeline.

Giai đoạn 3: Upload lên Google Cloud Storage (`gcs_uploader.py`)

Sau khi thu thập xong, module `gcs_uploader.py` đẩy toàn bộ file JSON và hình ảnh lên **Google Cloud Storage (GCS)** theo cấu trúc phân vùng ngày:

- JSON: `gs://incremental_raw/dt={crawl_date}/{page}/{file}.json`
- Hình ảnh: `gs://incremental_raw/dt={crawl_date}/images/{path}`

Mỗi lần chạy hàng tuần sử dụng một prefix `dt=` riêng biệt (ngày UTC hiện tại), đảm bảo **không bao giờ ghi đè** dữ liệu của các lần chạy trước. Cơ chế này hỗ trợ incremental loading ở tầng tiếp theo.

2.2.3 Điều phối tự động bằng Temporal

Toàn bộ ba giai đoạn trên được điều phối tự động bằng **Temporal** — một hệ thống workflow orchestration thay thế Airflow. Pipeline chạy theo lịch hàng tuần với workflow `WeeklyCrawlWorkflow`:

1. `crawl_links` → thu thập danh sách URL
2. `scrape_details` → thu thập chi tiết từng xe

3. `upload_gcs` → upload dữ liệu lên GCS

Temporal cung cấp cơ chế retry tự động với backoff policy riêng cho mỗi activity (ví dụ: scrape retry tối đa 2 lần vì tốn tài nguyên, upload retry tối đa 3 lần). Nếu một activity thất bại vượt quá giới hạn retry, workflow dừng lại (fail-stop) mà không ảnh hưởng đến dữ liệu đã thu thập thành công.

Crawler worker chạy trên **GCE VM** (cùng VM với Temporal server), sử dụng Chrome và Xvfb. Pipeline worker (dbt, embeddings) chạy trong Docker container trên cùng VM.

2.3 Xử lý và chuyển đổi dữ liệu (dbt Pipeline)

Sau khi dữ liệu thô được upload lên GCS, quá trình xử lý tiếp tục với hai bước: nạp dữ liệu vào database (load bronze) và chuyển đổi dữ liệu bằng dbt.

2.3.1 Nạp dữ liệu vào tầng Bronze (`load_bronze`)

Module `bronze.py` thực hiện nạp dữ liệu từ GCS vào bảng `bronze.raw_listings` trên PostgreSQL (AlloyDB):

1. **Scan GCS:** Liệt kê tất cả file `.json` trong prefix `dt={crawl_date}/` (hoặc toàn bộ bucket nếu chạy backfill).
2. **Download và parse song song:** Sử dụng `ThreadPoolExecutor` (16 worker) để tải và phân tích JSON đồng thời. Mỗi file được tính `SHA-256 hash` làm khóa idempotent, đồng thời trích xuất các trường quan trọng từ payload (`VIN`, `car_model_slug`, `crawled_at`).
3. **Bulk insert:** Chèn hàng loạt vào bảng bronze (batch 500 hàng) với mệnh đề `ON CONFLICT (file_hash) DO NOTHING` — đảm bảo **idempotent**: chạy lại cùng dữ liệu không tạo bản ghi trùng.

2.3.2 Chuyển đổi dữ liệu bằng dbt (Medallion Architecture)

Sau khi dữ liệu đã nằm trong tầng bronze, **dbt** (data build tool) tự động chuyển đổi qua ba tầng:

Staging (Silver Staging)

Tầng staging giải quyết hai vấn đề chính:

- **Giải trùng lặp:** Model `stg_raw_latest` sử dụng window function `ROW_NUMBER() OVER (PARTITION BY vin ORDER BY crawl_date DESC)` để giữ lại bản ghi mới nhất cho mỗi VIN. Điều này quan trọng vì bronze là append-only — cùng một xe có thể được crawl nhiều lần.
- **Trích xuất và flatten JSON:** Các model staging (`stg_listings`, `stg_sellers`, `stg_car_models`, `stg_listing_features`, `stg_listing_images`, `stg_model_reviews`) trích xuất dữ liệu từ cột `payload` (JSONB) của bronze, phẳng hóa cấu trúc JSON lồng nhau thành các trường quan hệ riêng biệt. Ví dụ: `payload->'post'->'basic_desc'->'VIN'` được trích thành cột `vin`.

Silver (Mô hình chiều – Dimensional Model)

Tầng Silver tổ chức dữ liệu theo mô hình 3NF (Third Normal Form) gồm các bảng fact và dimension:

- **Bảng Fact:**
 - `fact_listing`: Mỗi hàng là một bài đăng xe (incremental, delete+insert). Chứa khóa ngoại đến các bảng dimension, cùng các metric chính (giá, quãng đường, MPG, v.v.).
 - `fact_model_rating`: Điểm đánh giá tổng hợp theo dòng xe.
 - `fact_model_review`: Nhận xét chi tiết của từng người dùng.
- **Bảng Dimension:**
 - `dim_car_model`: Hãng xe, dòng xe, slug.
 - `dim_seller`: Tên đại lý, địa chỉ, thông tin liên hệ.
 - `dim_feature`: Danh mục tính năng (tên, nhóm).

- `dim_listing_image`: Hình ảnh xe (URL, thứ tự).

- **Bảng Bridge:**

- `bridge_listing_feature`: Quan hệ nhiều-nhiều giữa bài đăng và tính năng.

Gold (Bảng phục vụ ứng dụng)

Tầng Gold là các bảng denormalized (phi chuẩn hóa có chủ đích) tối ưu cho truy vấn từ backend API:

- `gold.vehicles`: Bảng chính — merge by VIN (incremental strategy). Mỗi hàng chứa toàn bộ thông tin cần thiết cho frontend: giá, hãng, dòng, `body_type`, màu sắc, hình ảnh chính, số lượng tính năng, điểm đánh giá dòng xe. Trường `vehicle_id` là alias của `vin` để tương thích ngược với backend.
- `gold.vehicle_price_history`: Lịch sử biến động giá xe theo ngày crawl, cho phép theo dõi xu hướng giá.
- `gold.car_models`, `gold.sellers`, `gold.reviews`: Dữ liệu tổng hợp theo dòng xe, người bán, và nhận xét.
- `gold.vehicle_features`, `gold.vehicle_images`: Tính năng và hình ảnh chi tiết.

2.3.3 Refresh materialized view và tính toán ML

Toàn bộ pipeline được điều phối bởi `WeeklyPipelineWorkflow` — một workflow cha chuỗi (chain) ba workflow con theo thứ tự **fail-stop** (nếu bước trước thất bại thì không chạy bước sau):

1. `WeeklyCrawlWorkflow`: `crawl_links` → `scrape_details` → `upload_gcs` (chạy trên `CRAWL_TASK_QUEUE`, host machine).
2. `TransformWorkflow`: `ensure_partition` → `load_bronze` → `dbt_build` → `refresh_matviews` (chạy trên `PIPELINE_TASK_QUEUE`, Docker container).
3. `MLWorkflow`: `compute_item_similarity` || `embed_vehicles` || `embed_chatbot_vehicles` (chạy song song trên `PIPELINE_TASK_QUEUE`).

Sau khi dbt build hoàn tất, các bước ML thực hiện:

- `refresh_matviews`: Làm mới các materialized view phục vụ hệ thống đề xuất (`mv_popular_vehicles`, `mv_rating`).
- `compute_item_similarity`: Tính ma trận cosine similarity giữa các xe dựa trên đặc trưng (giá, quãng đường, đánh giá, v.v.), lưu vào `gold.item_similarity` phục vụ Collaborative Filtering.
- `embed_vehicles`: Tạo vector embedding cho từng xe bằng OpenAI `text-embedding-3-large` (3072 chiều), lưu vào Qdrant collection `car_chatbot_vectors` — phục vụ Vector Recall trong hệ thống đề xuất.
- `embed_chatbot_vehicles`: Chia nhỏ (chunk) thông tin xe thành các đoạn văn bản, tạo embedding và lưu vào Qdrant collection `car_vectorize` — phục vụ chatbot RAG.

Lưu ý quan trọng: Hai collection Qdrant phục vụ hai mục đích hoàn toàn khác nhau — nhằm lẫn giữa chúng sẽ phá vỡ hệ thống đề xuất hoặc chatbot. Ba bước `compute_item_similarity`, `embed_vehicles` và `embed_chatbot_vehicles` chạy **song song** (parallel) nhờ Temporal, giảm thời gian xử lý tổng thể.

2.3.4 Tổng kết luồng xử lý dữ liệu

Toàn bộ pipeline từ thu thập đến sẵn sàng phục vụ ứng dụng được tóm tắt như sau:

Bước	Mô tả	Công nghệ
1	Thu thập URL bài đăng	SeleniumBase UC mode
2	Thu thập chi tiết xe + dealer	SeleniumBase + BeautifulSoup
3	Upload dữ liệu thô lên GCS	Google Cloud Storage
4	Nạp vào bronze (JSONB)	Python + psycopg2
5	Staging: dedup + flatten JSON	dbt (SQL view)
6	Silver: mô hình chiều 3NF	dbt (SQL table)
7	Gold: bảng denormalized	dbt (incremental merge)
8	Refresh materialized views	SQL
9	Cosine similarity matrix	Python (scikit-learn)
10	Vector embeddings	OpenAI + Qdrant

Pipeline chạy hoàn toàn tự động hàng tuần qua Temporal `WeeklyPipelineWorkflow`, đảm bảo dữ liệu luôn được cập nhật mà không cần can thiệp thủ công.

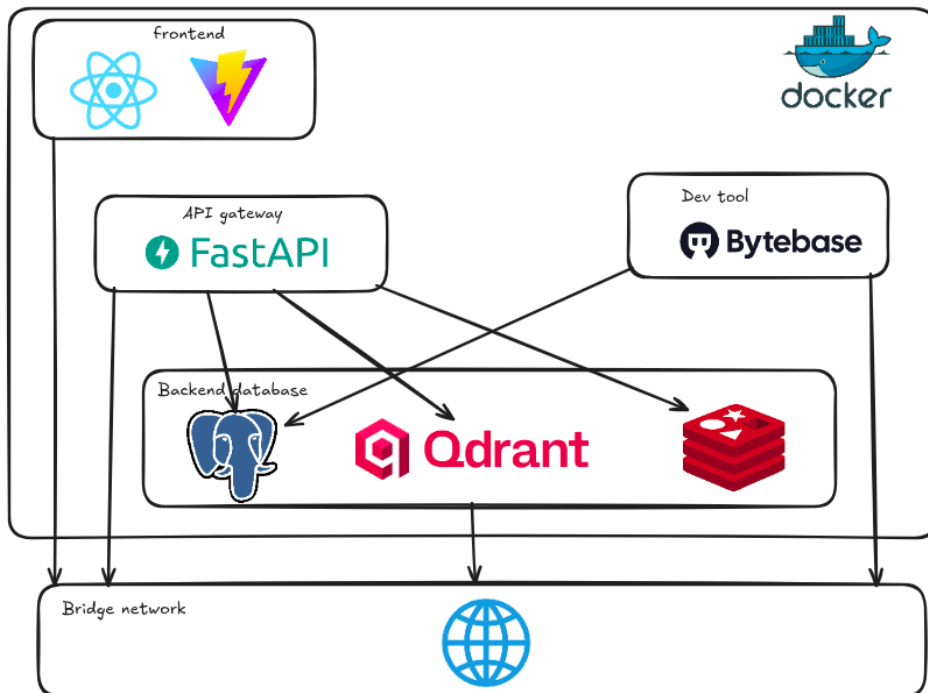
3 BACKEND VÀ HỆ THỐNG ĐỀ XUẤT

3.1 Giới thiệu

Hệ thống backend đóng vai trò là trung tâm xử lý chính của ứng dụng *Car Recommendation System*, chịu trách nhiệm tiếp nhận yêu cầu từ người dùng, xử lý nghiệp vụ, truy xuất dữ liệu và trả về kết quả gợi ý phù hợp. Backend hoạt động như một lớp trung gian, kết nối giữa giao diện người dùng (frontend) và các hệ thống lưu trữ dữ liệu phía sau, đảm bảo tính nhất quán, hiệu năng và khả năng mở rộng của toàn bộ hệ thống.

Backend được xây dựng dựa trên framework **FastAPI** với ngôn ngữ **Python**, cho phép phát triển các API theo kiến trúc RESTful với hiệu năng cao, khả năng mở rộng tốt và hỗ trợ tài liệu API tự động thông qua OpenAPI/Swagger. FastAPI được lựa chọn nhờ khả năng xử lý bất đồng bộ (asynchronous), phù hợp với các hệ thống cần phục vụ nhiều request đồng thời.

Hệ thống sử dụng **PostgreSQL** làm cơ sở dữ liệu quan hệ để lưu trữ các dữ liệu có cấu trúc như thông tin xe, người bán, giao dịch và metadata liên quan. PostgreSQL đảm bảo tính toàn vẹn dữ liệu, hỗ trợ các ràng buộc quan hệ và khả năng truy vấn phức tạp. Bên cạnh đó, **Qdrant** được sử dụng như một vector database, phục vụ cho bài toán tìm kiếm và gợi ý dựa trên vector embedding, cho phép hệ thống thực hiện các phép so sánh tương đồng (similarity search) hiệu quả trên không gian nhiều chiều. Để tối ưu hiệu năng và giảm tải cho backend cũng như database, **Redis** được sử dụng làm hệ thống cache, lưu trữ các kết quả truy vấn thường xuyên hoặc dữ liệu tạm thời.

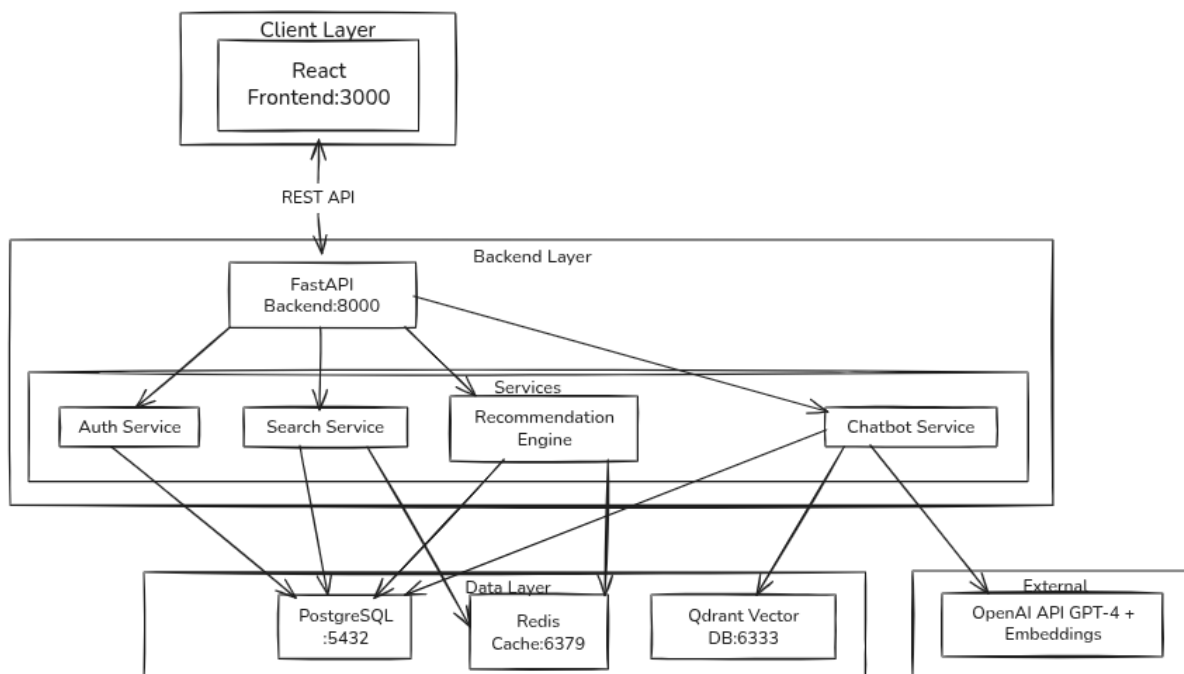


Hình 3.1: Kiến trúc tổng thể hệ thống backend

Hệ thống được thiết kế theo mô hình **microservices**, trong đó mỗi thành phần đảm nhiệm một vai trò riêng biệt và có thể triển khai, mở rộng độc lập. Frontend Service chạy trên cổng **3000**, chịu trách nhiệm hiển thị giao diện và tương tác trực tiếp với người dùng. Backend API chạy trên cổng **8000**, cung cấp các endpoint để xử lý nghiệp vụ và điều phối dữ liệu giữa các dịch vụ. PostgreSQL database hoạt động trên cổng **5432**, Qdrant vector database trên cổng **6333**, và Redis cache trên cổng **6379**.

Toàn bộ các service được đóng gói và triển khai bằng **Docker**, giao tiếp với nhau thông qua **Docker bridge network**. Cách tiếp cận này giúp hệ thống dễ dàng triển khai trên nhiều môi trường khác nhau, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường triển khai, đồng thời tạo tiền đề thuận lợi cho việc mở rộng hoặc tích hợp thêm các dịch vụ mới trong tương lai.

3.2 Kiến trúc tổng quan



Hình 3.2: Kiến trúc tổng quan hệ thống

Như minh họa trong Hình 3.2, hệ thống được thiết kế theo kiến trúc phân tầng (layered architecture) kết hợp với mô hình microservices nhằm đảm bảo tính tách biệt chức năng, khả năng mở rộng và dễ bảo trì. Toàn bộ hệ thống được chia thành ba lớp chính: *Client Layer*, *Backend Layer* và *Data Layer*.

Client Layer bao gồm ứng dụng frontend được xây dựng bằng React, chạy trên cổng 3000. Lớp này chịu trách nhiệm hiển thị giao diện người dùng, thu thập dữ liệu đầu vào và gửi các yêu cầu đến backend thông qua các REST API. Client không xử lý logic nghiệp vụ phức tạp mà tập trung vào trải nghiệm người dùng và tương tác.

Backend Layer đóng vai trò trung tâm điều phối và xử lý nghiệp vụ của hệ thống. Backend được xây dựng trên nền tảng FastAPI, chạy trên cổng 8000, cung cấp các endpoint RESTful cho frontend. Bên trong backend, các chức năng được chia thành nhiều service độc lập nhằm tuân thủ nguyên tắc single responsibility:

- *Authentication Service*: quản lý xác thực người dùng, đăng nhập, phân quyền và kiểm soát truy cập.
- *Search Service*: xử lý các yêu cầu tìm kiếm xe dựa trên tiêu chí lọc, từ khóa và các điều kiện truy vấn.
- *Recommendation Engine*: thực hiện logic gợi ý xe dựa trên hành vi người dùng và độ tương đồng dữ liệu, kết hợp truy vấn vector embedding.
- *Chatbot Service*: cung cấp khả năng tư vấn tự động, hỗ trợ người dùng thông qua hội thoại và tích hợp với mô hình ngôn ngữ lớn.

Các service trong backend có thể giao tiếp với nhau thông qua các lời gọi nội bộ và cùng truy cập các nguồn dữ liệu phía dưới, giúp hệ thống dễ dàng mở rộng hoặc bổ sung thêm service mới trong tương lai.

Data Layer chịu trách nhiệm lưu trữ và truy xuất dữ liệu, bao gồm nhiều thành phần chuyên biệt cho từng loại dữ liệu. PostgreSQL được sử dụng làm cơ sở dữ liệu quan hệ để lưu trữ dữ liệu có cấu trúc như thông tin xe, người dùng và người bán. Redis được sử dụng làm hệ thống cache, giúp lưu trữ dữ liệu tạm

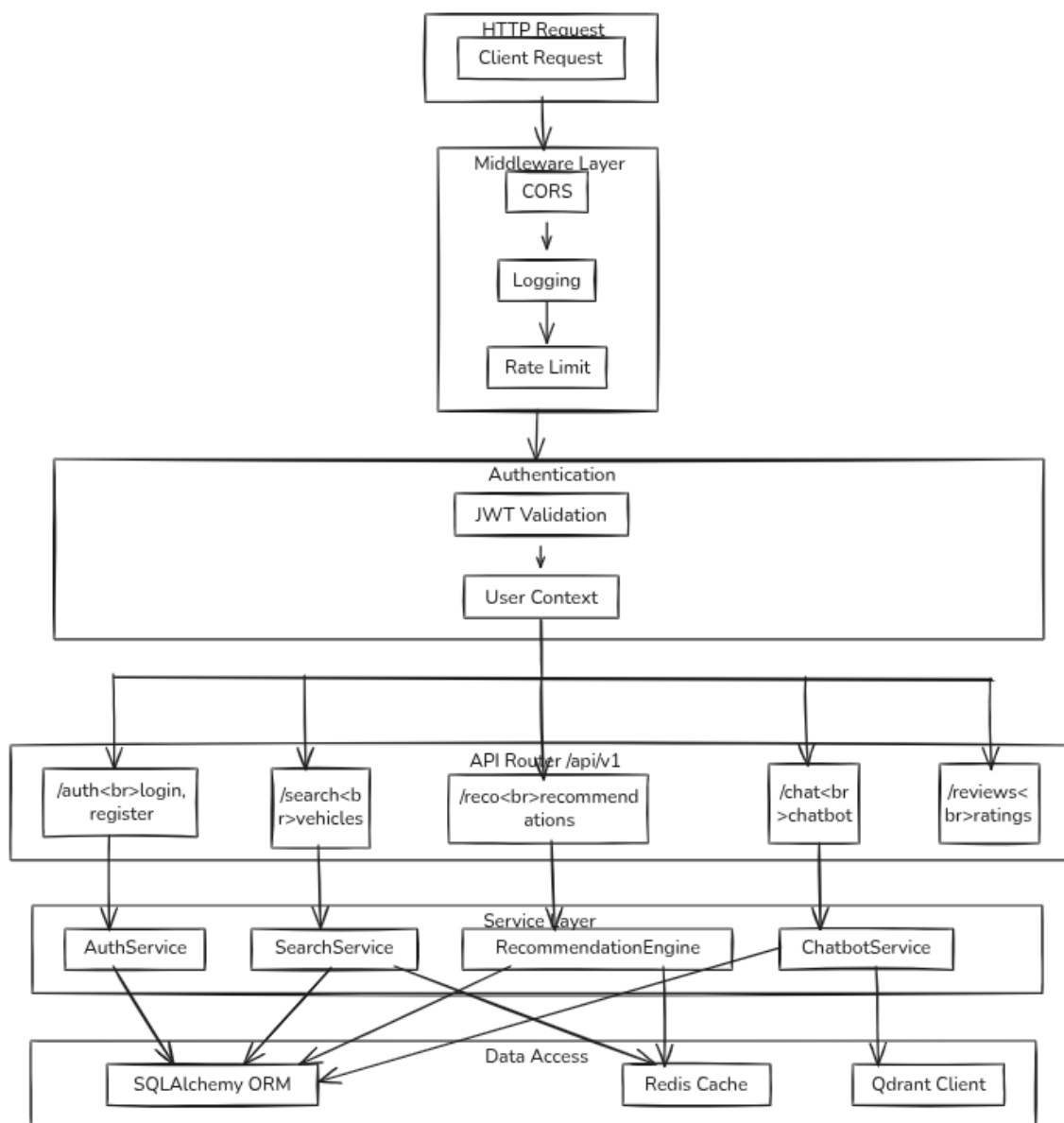
thời và kết quả truy vấn thường xuyên nhằm giảm độ trễ và tăng hiệu năng. Qdrant đóng vai trò là vector database, phục vụ cho các bài toán tìm kiếm và gợi ý dựa trên độ tương đồng của vector embedding.

Ngoài ra, hệ thống còn tích hợp với các dịch vụ bên ngoài như OpenAI API để hỗ trợ chatbot và tạo embedding phục vụ cho Recommendation Engine. Việc tách biệt các dịch vụ bên ngoài giúp hệ thống linh hoạt trong việc thay thế hoặc mở rộng các nhà cung cấp khác trong tương lai.

Tất cả các thành phần trong hệ thống được đóng gói và triển khai thông qua Docker, giao tiếp với nhau thông qua Docker bridge network. Cách tiếp cận này giúp đảm bảo tính nhất quán giữa các môi trường triển khai, đồng thời tạo nền tảng cho việc mở rộng hệ thống lên các hạ tầng orchestration phức tạp hơn như Kubernetes khi cần thiết.

3.3 Kiến trúc API

Backend API được tổ chức theo mô hình phân tầng với bốn lớp chính.



Hình 3.3: Kiến trúc phân tầng của Backend API

Middleware Layer là lớp đầu tiên tiếp nhận request, thực hiện kiểm tra CORS để cho phép cross-origin requests từ frontend, ghi log mọi request để theo dõi và debug, và áp dụng rate limiting để ngăn chặn lạm

dụng API.

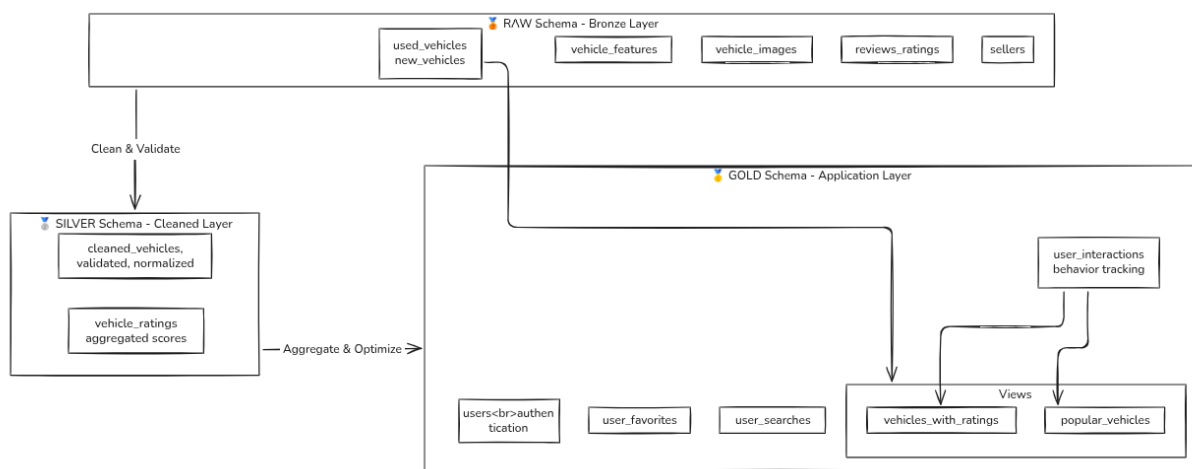
Authentication Layer xác thực JWT token trong header Authorization, giải mã và kiểm tra tính hợp lệ của token, sau đó gắn thông tin user vào context để các lớp sau sử dụng.

Router Layer định tuyến request đến endpoint tương ứng bao gồm auth cho đăng nhập và đăng ký, search cho tìm kiếm xe, reco cho gợi ý, chat cho chatbot, và reviews cho đánh giá. Mỗi router thực hiện validation dữ liệu đầu vào theo schema định nghĩa sẵn.

Service Layer chứa business logic của từng tính năng. AuthService xử lý hash password và tạo token. SearchService xây dựng query động và cache kết quả. RecommendationEngine tính toán similarity và rank gợi ý. ChatbotService thực hiện RAG workflow với vector database.

3.4 Thiết kế Database

Hệ thống sử dụng kiến trúc multi-schema theo mô hình **Medallion Architecture** với ba tầng dữ liệu: Bronze, Silver và Gold. Các bảng trong Silver và Gold được tạo và quản lý bởi **dbt** (data build tool), ngoại trừ các bảng user-domain do ứng dụng FastAPI ghi trực tiếp.



Hình 3.4: Kiến trúc multi-schema database

Bronze Schema lưu trữ dữ liệu thô trực tiếp từ quá trình crawling, chỉ gồm một bảng duy nhất `bronze.raw_listings`. Mỗi hàng chứa toàn bộ file JSON crawled dưới dạng cột `payload` (JSONB), kèm metadata như `file_hash` (SHA-256, khóa idempotent), `gcs_path`, `vin`, `crawl_date`. Bảng này là **append-only**: cùng một VIN crawl lại sẽ tạo hàng mới (khác `file_hash`), không ghi đè. Việc chọn bản ghi mới nhất được giải quyết ở tầng staging.

Silver Schema do dbt tạo, tổ chức theo mô hình 3NF (Third Normal Form):

- **Bảng Fact:** `fct_listing` (bài đăng xe, incremental delete+insert), `fct_model_rating` (điểm đánh giá tổng hợp), `fct_model_review` (nhận xét chi tiết).
- **Bảng Dimension:** `dim_car_model` (hãng, dòng xe), `dim_seller` (thông tin đại lý), `dim_feature` (danh mục tính năng), `dim_listing_image` (hình ảnh xe).
- **Bảng Bridge:** `bridge_listing_feature` (quan hệ nhiều-nhiều giữa bài đăng và tính năng).

Gold Schema gồm hai nhóm bảng:

- **Bảng do dbt tạo (marts):** `vehicles` (denormalized, merge by VIN — bảng chính phục vụ frontend), `vehicle_price_history` (partitioned by month), `car_models`, `sellers`, `reviews`, `vehicle_features`, `vehicle_images`. Kèm theo các materialized view: `mv_popular_vehicles` và `mv_rating`.
- **Bảng do ứng dụng tạo (user-domain):** `users`, `user_interactions` (hành vi người dùng phục vụ recommendation), `user_favorites`, `user_searches`, `user_reviews`, `chat_sessions`, `chat_messages`, `item_similarity` (ma trận CF precomputed).

Lưu ý thiết kế: Trường `vehicle_id` trong các bảng user-domain là kiểu TEXT (VIN), **không có foreign key** đến `gold.vehicles`. Lý do: bảng `gold.vehicles` được dbt DROP/CREATE khi chạy full-refresh; một FK cross-schema sẽ phá vỡ quá trình rebuild. Tính toàn vẹn tham chiếu được đảm bảo thay thế bằng dbt test `assert_no_orphan_interactions`.

3.5 Data Pipeline

Quy trình chuyển đổi dữ liệu từ nguồn thô (crawled JSON trên GCS) đến trạng thái sẵn sàng phục vụ ứng dụng được điều phối hoàn toàn tự động bởi **Temporal** thông qua `WeeklyPipelineWorkflow`. Chi tiết từng giai đoạn đã được trình bày tại Chương 2 (Quá trình thu thập và xử lý dữ liệu); phần này tóm tắt luồng xử lý từ góc nhìn backend.

Pipeline gồm ba workflow con chạy tuần tự (fail-stop):

1. **WeeklyCrawlWorkflow:** Thu thập dữ liệu từ Cars.com và upload JSON lên Google Cloud Storage theo cấu trúc phân vùng ngày `dt=YYYY-MM-DD/`.
2. **TransformWorkflow:** Nạp JSON vào `bronze.raw_listings` (idempotent qua `file_hash`), sau đó chạy dbt build để chuyển đổi qua các tầng staging → silver → gold theo Medallion Architecture.
3. **MLWorkflow:** Chạy song song ba tác vụ — tính ma trận `item_similarity` (cosine similarity), tạo vector embedding cho hệ thống đề xuất (`car_chatbot_vectors`), và tạo chunked embedding cho chatbot RAG (`car_vectorize`).

Toàn bộ pipeline chạy tự động hàng tuần, đảm bảo dữ liệu luôn được cập nhật mà không cần can thiệp thủ công. Tính chất **incremental** và **idempotent** cho phép chạy lại an toàn: bronze dedup bằng `file_hash`, gold merge by VIN, Qdrant upsert bằng `point_id = uuid5(vin)`.

3.6 Hệ thống Authentication

Hệ thống sử dụng JWT (JSON Web Token) cho xác thực người dùng.

Khi người dùng đăng ký, hệ thống nhận username, email và password từ frontend. Password được hash bằng bcrypt với salt ngẫu nhiên trước khi lưu vào database. Sau đó tạo JWT token chứa `user_id` với thời hạn 30 phút và trả về cho frontend.

Khi người dùng đăng nhập, hệ thống query user từ database theo username, verify password bằng cách so sánh hash, nếu đúng thì tạo và trả về JWT token mới.

Với mỗi request cần xác thực, frontend gửi token trong header Authorization. Backend extract token, decode và verify signature với `SECRET_KEY`, kiểm tra expiration time, sau đó query user từ database và gắn vào request context.

3.7 Caching và Tối ưu hiệu năng

Để đảm bảo hiệu năng và khả năng mở rộng, hệ thống áp dụng cơ chế caching nhiều tầng với **Redis** làm trung tâm, kết hợp tối ưu truy vấn ở tầng cơ sở dữ liệu.

3.7.1 Caching bằng Redis

Hệ thống sử dụng Redis với TTL mặc định là **1 giờ** (`CACHE_TTL = 3600`), bao gồm các lớp cache sau:

- *API Response Cache:* Lưu trữ kết quả của các truy vấn tìm kiếm phổ biến — các request có cùng tập tham số lọc sẽ được phục vụ trực tiếp từ cache, giảm tải cho PostgreSQL và Qdrant.
- *Recommendation Cache:* Lưu kết quả gợi ý xe cho từng người dùng. Cache được chủ động *invalidate* khi hệ thống ghi nhận tương tác mới (click, save, contact, v.v.), đảm bảo kết quả gợi ý phản ánh hành vi mới nhất.

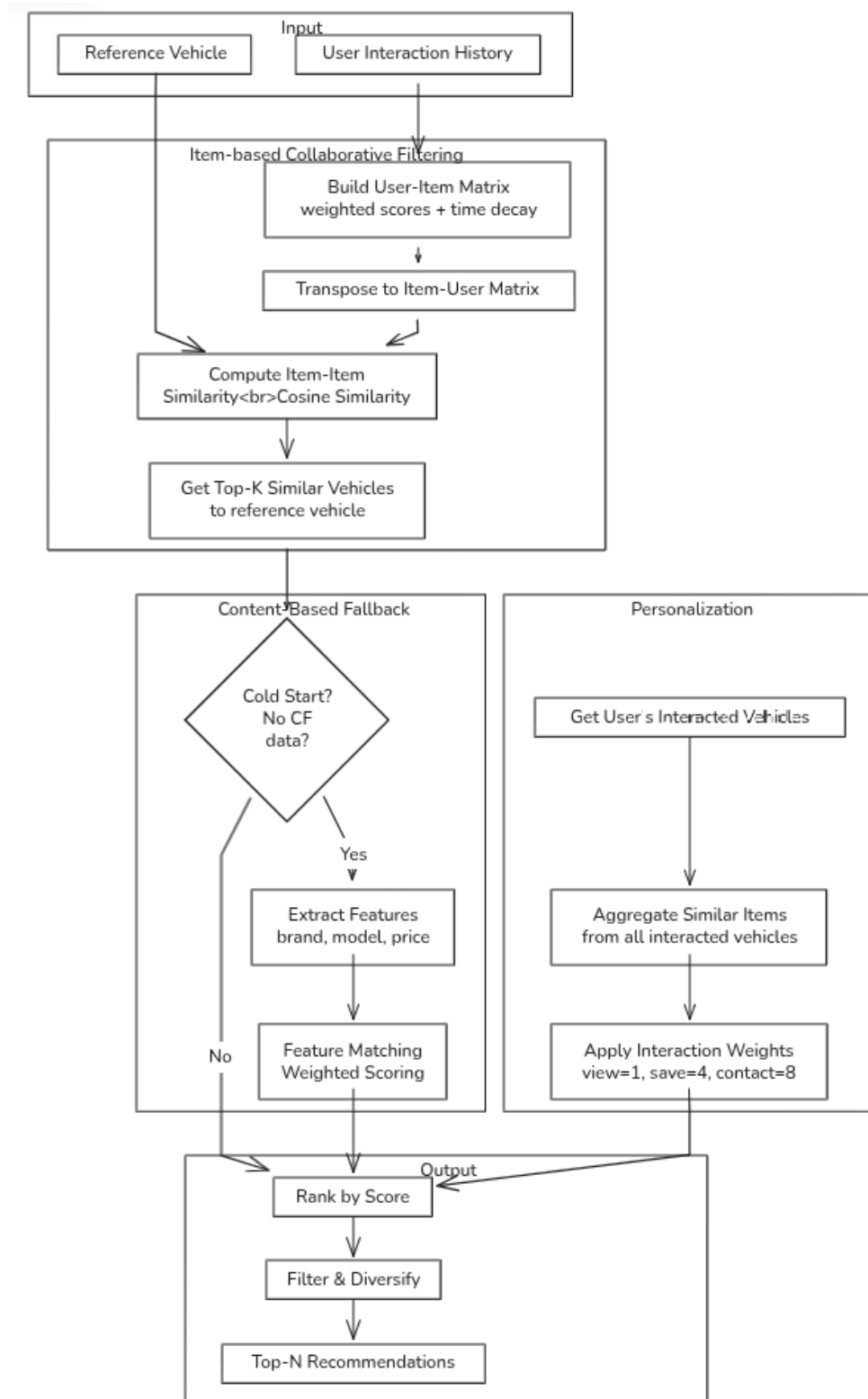
3.7.2 Tối ưu truy vấn Database

Các chỉ mục (index) được tạo trên các cột thường xuyên sử dụng trong điều kiện lọc và join. Cụ thể, bảng `gold.user_interactions` có index trên `user_id`, `vehicle_id`, `interaction_type` và `created_at`. Bảng `bronze.raw_listings` có GIN index trên cột `payload` (JSONB) để tăng tốc truy vấn JSON.

Đối với các truy vấn tổng hợp phức tạp, hệ thống sử dụng **materialized views** — `mv_popular_vehicles` (xe phổ biến theo lượt tương tác) và `mv_rating` (điểm đánh giá trung bình) — được refresh tự động mỗi lần pipeline chạy. Cách tiếp cận này giúp giảm chi phí tính toán tại thời điểm truy vấn và đảm bảo hiệu năng ổn định cho các API thường xuyên được gọi.

3.8 Hệ thống Recommendation

Recommendation Engine được thiết kế theo kiến trúc **3-stage hybrid pipeline**, xử lý tuần tự qua ba giai đoạn: candidate generation, ranking và re-ranking. Engine được khởi tạo mới cho mỗi request (stateless) — không fit model trong request, mà sử dụng các tín hiệu precomputed (`item_similarity`, vector embeddings) đã tính sẵn bởi pipeline ML.



Hình 3.5: Luồng hoạt động của Recommendation Engine

3.8.1 Stage 1 — Candidate Generation

Giai đoạn đầu tiên thu thập một tập ứng viên rộng từ **4 recaller chạy song song**, mỗi recaller khai thác một nguồn tín hiệu khác nhau:

- **CollaborativeRecaller:** Item-based Collaborative Filtering sử dụng bảng `gold.item_similarity` đã precomputed. Với mỗi xe mà user đã tương tác (seed), hệ thống tra cứu K xe láng giềng tương tự nhất. Điểm số được tính có trọng số theo loại tương tác và time decay:

$$\text{score}_{\text{neighbor}} = \sum_{\text{seed}} w_{\text{type}} \cdot e^{-\lambda \cdot \text{days}} \cdot \text{similarity}$$

trong đó trọng số tương tác: view=1.0, click=2.0, compare=3.0, save/favorite=4.0, contact/inquiry=8.0; $\lambda = 0.05$ (half-life ≈ 14 ngày).

- **ContentRecaller:** Tìm xe tương tự dựa trên đặc tả kỹ thuật (brand, model, fuel_type, transmission, drivetrain) và khung giá ($\pm 30\%$ giá xe seed). Sử dụng SQL query trực tiếp trên `gold.vehicles`, tính điểm tương đồng theo rule-based scoring:

$$\text{score} = 2.0 \cdot [\text{brand}] + 1.5 \cdot [\text{model}] + 0.5 \cdot [\text{fuel}] + 0.5 \cdot [\text{trans}] + 0.3 \cdot [\text{drive}] + 1.0 \cdot [\text{price_in_band}]$$

- **VectorRecaller:** Tìm kiếm ngữ nghĩa (semantic similarity) trên Qdrant collection `car_chatbot_vectors`. Truy xuất vector embedding đã lưu của xe seed, sau đó tìm các vector láng giềng gần nhất trong không gian 3072 chiều.
- **PopularityRecaller:** Xe phổ biến toàn hệ thống từ materialized view `mv_popular_vehicles`. Ưu tiên cùng brand với seed vehicle. Đây cũng là **fallback cho cold-start**: người dùng mới hoặc khách (guest) không có lịch sử tương tác sẽ nhận gợi ý hoàn toàn từ recaller này.

Các recaller hoạt động theo cơ chế **fail-soft**: nếu một nguồn dữ liệu không khả dụng (ví dụ: Qdrant tạm ngừng, matview chưa tồn tại), recaller trả về tập rỗng thay vì raise exception, đảm bảo hệ thống vẫn hoạt động với các nguồn còn lại.

3.8.2 Stage 2 — Ranking

Sau khi thu thập ứng viên, module `FeatureAssembler` hợp nhất kết quả từ 4 recaller, bổ sung thuộc tính xe từ `gold.vehicles` (brand, model, price, rating, crawled_at), và tính toán thêm 3 derived features:

- **price_fit:** Mức độ phù hợp với ngân sách user (= giá trung bình các xe đã xem). Giá càng gần budget, điểm càng cao (1.0 = khớp hoàn toàn, giảm tuyến tính khi lệch).
- **model_rating:** Điểm đánh giá dòng xe từ Cars.com (0–5, normalize về 0–1).
- **recency:** Độ mới của bài đăng, sử dụng exponential decay với half-life ≈ 30 ngày.

`WeightedLinearRanker` tính điểm tổng cho mỗi ứng viên bằng công thức:

$$\text{rank_score} = \sum_{i=1}^7 w_i \cdot f_i$$

với 7 features và trọng số cấu hình trong `reco_config.yaml`:

Feature	Weight	Nguồn
cf_score	3.0	CollaborativeRecaller
vector_score	2.0	VectorRecaller (Qdrant)
content_score	1.5	ContentRecaller (SQL)
popularity	1.0	PopularityRecaller (matview)
price_fit	1.0	Derived (budget proximity)
model_rating	0.8	Cars.com consumer rating
recency	0.5	Listing freshness

CF warmup: Trọng số `cf_score` được nhân thêm hệ số `cf_scale` $\in [0, 1]$ tỉ lệ thuận với số tương tác của user. Khi user mới (0 interactions), `cf_scale` = 0 và CF không đóng góp vào điểm. Khi user đạt ngưỡng

20 interactions, `cf_scale = 1.0` và CF phát huy tối đa. Cơ chế này giúp hệ thống chuyển dịch tự nhiên từ content/popularity-driven sang collaborative-driven khi tích lũy đủ dữ liệu hành vi.

Ranker được thiết kế theo mô hình **weighted-linear** (thay vì black-box model) để đảm bảo tính giải thích được (explainability): có thể chỉ ra chính xác tín hiệu nào đẩy một xe lên cao trong danh sách.

3.8.3 Stage 3 — Re-ranking

MMRReranker (Maximal Marginal Relevance) cân bằng giữa **relevance** và **diversity** trong kết quả cuối cùng:

$$\text{MMR} = \lambda \cdot \text{relevance} - (1 - \lambda) \cdot \max_{\text{đã chọn}} \text{similarity}$$

với $\lambda = 0.7$ (nghiêng về relevance nhưng vẫn phạt trùng lặp). Similarity giữa hai xe được tính nhanh theo brand (0.5) và model (0.5) mà không cần embedding.

Ngoài ra, hệ thống áp dụng **hard caps** để tránh kết quả đơn điệu:

- Tối đa **3 xe** cùng brand trong danh sách cuối.
- Tối đa **2 xe** cùng car_model.

Khi tất cả ứng viên còn lại đều vi phạm caps, hệ thống *relax* ràng buộc và chọn theo relevance để đảm bảo đủ `top_k` kết quả (mặc định 20).

3.8.4 Tóm tắt luồng xử lý

Stage	Module	Mô tả
1	candidates.py	4 recaller song song $\rightarrow$ tập ứng viên (<code>pool_size $\leq$ 300</code>)
2	features.py + ranker.py	Bổ sung 7 features, tính <code>rank_score</code> tuyến tính
3	reranker.py	MMR diversity + brand/model caps $\rightarrow$ <code>top_k = 20</code>

Toàn bộ trọng số và ngưỡng được cấu hình tập trung trong file `reco_config.yaml`, không có magic numbers trong code. Kiến trúc này cho phép điều chỉnh chiến lược gợi ý chỉ bằng cách thay đổi file YAML mà không cần sửa code.

4 XÂY DỰNG CHATBOT

4.1 Mục tiêu và phạm vi tư vấn

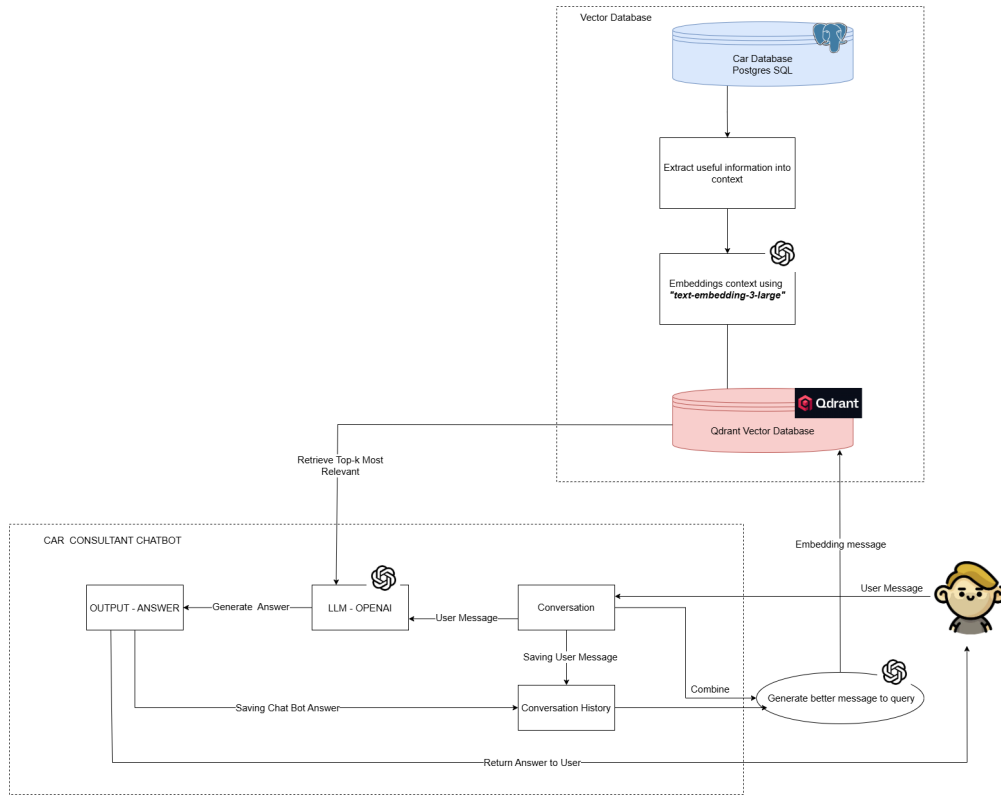
Dựa trên bộ dữ liệu thu thập từ *Cars.com*, nhóm đặt mục tiêu xây dựng chatbot có khả năng hỗ trợ người dùng đặt câu hỏi và nhận tư vấn một cách tự nhiên, đồng thời đảm bảo câu trả lời bám sát nguồn thông tin thực tế hiện có trong dữ liệu. Chatbot được thiết kế để duy trì ngữ cảnh hội thoại, ghi nhớ các câu hỏi trước đó và khai thác thông tin đã cung cấp nhằm hiểu rõ hơn hành vi cũng như nhu cầu của người dùng trong suốt quá trình tương tác.

Phạm vi tư vấn:

- **Gợi ý xe theo nhu cầu:** Người dùng cung cấp các tiêu chí như khoảng giá, loại xe, hãng xe, màu sắc, các tính năng cần thiết; chatbot trả về danh sách các bài đăng phù hợp nhất và tóm tắt các thông tin quan trọng của từng lựa chọn.
- **So sánh các lựa chọn:** Chatbot hỗ trợ so sánh nhiều mẫu xe trong cùng tầm giá dựa trên thông số và tính năng đã thu thập, giúp người dùng đánh giá ưu/nhược điểm để lựa chọn phù hợp.
- **Hỏi thông số kỹ thuật:** Chatbot trả lời câu hỏi về thông số kỹ thuật, tính năng an toàn, động cơ, MPG của một mẫu xe cụ thể.
- **Phân tích thống kê nền tảng:** Chatbot cung cấp thống kê tổng hợp như hãng phổ biến nhất, phân bố loại nhiên liệu, mô hình được đánh giá cao nhất.
- **Cung cấp hình ảnh minh họa:** Chatbot hiển thị hình ảnh của các mẫu xe được đề xuất thông qua URL từ dữ liệu.

4.2 Mô hình cơ sở xây dựng chatbot – Agentic RAG

Trong dự án này, nhóm sử dụng mô hình **RAG** (Retrieval-Augmented Generation) làm nền tảng, nhưng nâng cấp lên kiến trúc **Agentic RAG** sử dụng **LangGraph** để xử lý đa dạng loại câu hỏi. RAG kết hợp giữa **truy xuất thông tin** và **sinh ngôn ngữ**, tạo câu trả lời dựa trên dữ liệu thực tế thay vì chỉ dựa vào kiến thức sẵn có của mô hình.



Hình 4.1: Mô hình RAG tổng quát làm nền tảng lý thuyết cho hệ thống chatbot tư vấn xe

Điểm khác biệt so với RAG thuần túy là hệ thống được xây dựng trên **LangGraph StateGraph** với 12 nodes và **conditional routing** theo ý định câu hỏi. Mỗi request đi qua một đồ thị trạng thái **AgenticState** chứa: lịch sử hội thoại, câu hỏi hiện tại, context đã truy xuất, ý định phân loại, thông tin hồ sơ người dùng và danh sách xe kết quả.

4.3 Xây dựng Vector Database cho ChatBot

Vector database nhóm sử dụng trong dự án là **Qdrant**. Ứng dụng kết nối đến Qdrant thông qua **Qdrant-Client**. Chatbot sử dụng collection **car_vectorize** — lưu trữ thông tin xe dưới dạng **chunked documents** (chia nhỏ mô tả xe thành nhiều đoạn văn bản), phục vụ tìm kiếm ngữ nghĩa cho chatbot RAG.

Lưu ý quan trọng về hai Qdrant collection:

- **car_vectorize**: dành riêng cho **chatbot** (chunked docs, tìm kiếm ngữ nghĩa).
- **car_chatbot_vectors**: dành riêng cho **Recommendation Engine** (1 vector/VIN, VectorRecaller). Nhằm lẫn hai collection này sẽ phá vỡ cả chatbot lẫn hệ thống đề xuất.

Quá trình xây dựng vector database được thực hiện như sau:

- **Trích xuất và tạo nội dung mô tả xe**: Nhóm truy vấn SQL để lấy các trường thông tin quan trọng từ **gold.vehicles** (năm sản xuất, hãng xe, dòng xe, dáng xe, động cơ, nhiên liệu, màu sắc, giá cả, số km, tình trạng, địa chỉ, tính năng, nhận xét) và tổng hợp thành đoạn mô tả cho từng xe. Thông tin xe sau đó được **chia nhỏ thành các chunk** để tăng độ chính xác tìm kiếm ngữ nghĩa.
- **Khởi tạo collection**: Collection **car_vectorize** được cấu hình với **size = 3072** (số chiều embedding từ mô hình **text-embedding-3-large**) và **distance = COSINE** (thước đo tương đồng).
- **Nạp dữ liệu vào collection**: Dữ liệu được ingest theo từng **batch** để tối ưu hiệu năng. Mỗi PointStruct có:
 - **ID**: **uuid5(vin)** — ID xác định (deterministic), cho phép upsert idempotent.
 - **Vector**: embedding 3072 chiều của đoạn mô tả xe.
 - **Payload**: thông tin xe phục vụ lọc và hiển thị (VIN, năm, hãng, dòng xe, giá, số km, nhiên liệu,

màu sắc, vị trí, features, image_url, mô tả ngắn).

Việc dùng `uuid5(vin)` thay vì `sequential ID` đảm bảo rằng khi pipeline ML chạy lại hàng tuần, Qdrant thực hiện **upsert** (cập nhật nếu đã tồn tại) thay vì tạo bản sao, tránh trùng lặp dữ liệu.

4.4 Kiến trúc Agentic: LangGraph StateGraph

Thay vì một RAG chain tuyến tính, chatbot được xây dựng trên **LangGraph StateGraph** — một đồ thị trạng thái điều phối luồng xử lý theo ý định câu hỏi. Toàn bộ trạng thái hội thoại được lưu trong **AgenticState** (TypedDict) truyền qua các nodes.

4.4.1 Hồ sơ người dùng (User Profile Slot-Fill)

Mỗi phiên hội thoại duy trì một **UserProfile** riêng biệt theo `session_id`, lưu **in-memory** trong backend. Hồ sơ gồm hai nhóm thông tin:

- **Core Slots** (thông tin cốt lõi): `budget_max`, `body_type`, `fuel_type`, `brand`, `condition` — ba slots đầu tiên (`budget_max`, `body_type`, `fuel_type`) là **bắt buộc** để thực hiện truy xuất tư vấn.
- **Soft Preferences** (sở thích mềm): danh sách tính năng mong muốn (`features`), phong cách xe (`vibe`); cùng với `excluded_brands` và `viewed_models`.

Node `update_profile` chạy đầu tiên trong mỗi request, dùng LLM để trích xuất thông tin mới từ câu hỏi hiện tại và cập nhật vào hồ sơ. Nếu câu hỏi thuộc intent *vague* (nhu cầu chung) mà core slots chưa đủ, node `ask_slot` tự động sinh câu hỏi dẫn dắt để thu thập thêm thông tin, thay vì trả về kết quả chưa đủ cơ sở.

4.4.2 Phân loại ý định (Intent Routing)

Node `route_intent` sử dụng LLM với **structured output** (`IntentDecision`) để phân loại câu hỏi thành **7 loại intent**:

Intent	Mô tả	Ví dụ
<code>compare</code>	So sánh 2+ xe cụ thể	"Toyota Camry vs Honda Civic"
<code>analytics</code>	Thống kê nền tảng	"Hãng nào có nhiều xe nhất?"
<code>specs</code>	Thông số kỹ thuật 1 xe	"BMW X5 có những tính năng gì?"
<code>specific</code>	Tìm xe theo hãng/model	"Show me available BMW i5"
<code>vague</code>	Nhu cầu chung, không chỉ rõ xe	"Tôi cần xe gia đình \$40,000"
<code>chitchat</code>	Hội thoại liên quan đến xe	"Chào", "Cảm ơn"
<code>off_topic</code>	Ngoài phạm vi xe	"Thời tiết hôm nay thế nào?"

Kết quả phân loại còn trích xuất **brand** và **model** nếu có trong câu hỏi. Node `route_after_intent` sau đó định tuyến đến nhánh xử lý phù hợp.

4.5 Hệ thống Hybrid Retrieval và Sinh câu trả lời

4.5.1 Hybrid Retrieval

Với các intent `specific` và `vague` (sau khi đủ core slots), hệ thống thực hiện **hybrid retrieval** kết hợp hai nguồn song song:

- **SQL Hard-Filter** (`sql_search_cars`): Truy vấn trực tiếp trên `gold.vehicles` với điều kiện lọc cứng: `brand`, `model`, khoảng giá, loại nhiên liệu, tình trạng xe (`new/used`), năm sản xuất, `excluded brands`. Trả về tối đa 6 kết quả khớp chính xác.
- **Qdrant Semantic Search**: Tìm kiếm vector trên collection `car_vectorize` với câu hỏi đã được chuẩn hóa (`standalone question`). Lọc bỏ kết quả có `score > 1.3` (khoảng cách cosine quá xa). Trả về tối đa 5 kết quả ngữ nghĩa gần nhất.

Sau khi truy xuất, hệ thống tổng hợp thông tin từ `gold.vehicle_images` và `gold.vehicle_features` theo VIN để làm giàu context. Context cuối cùng có dạng:

[SQL Hard-Filter Matches] + [Semantic Matches]

4.5.2 Các nhánh xử lý chuyên biệt

Mỗi nhánh intent có pipeline retrieval và prompt template riêng:

- **compare_retrieve / compare_answer:** Trích xuất danh sách xe cần so sánh bằng `CompareExtraction` (structured output), truy vấn SQL từng xe, tổng hợp thông số đầy đủ. LLM sinh bảng so sánh side-by-side với verdict.
- **analytics_retrieve / analytics_answer:** Thực thi nhiều SQL query tổng hợp song song (brand counts, price stats, top models, fuel distribution, top features) trên `gold.vehicles` và `gold.vehicle_features`. LLM trình bày số liệu và insight.
- **spec_retrieve / spec_answer:** Lấy thông số kỹ thuật chi tiết (engine, drivetrain, MPG, transmission, fuel type) cùng features theo VIN. LLM trình bày có phân loại (Performance, Drivetrain, Safety, Comfort, Technology).
- **ask_slot:** Khi intent là *vague* nhưng thiếu core slots, sinh câu hỏi dẫn dắt tự nhiên để thu thập thông tin, có gợi ý cụ thể (ví dụ: SUV vs Sedan, Gasoline vs Hybrid vs Electric).
- **redirect_topic:** Khi intent là *off_topic*, lịch sự chuyển hướng cuộc trò chuyện về chủ đề xe.

4.5.3 Sinh câu trả lời (Generation)

Hệ thống sử dụng mô hình LLM **gpt-4o-mini** của OpenAI với thiết lập **temperature = 0.5**. Mỗi nhánh có **system prompt riêng** được thiết kế theo ngữ cảnh tư vấn cụ thể:

- **System prompt:** Mô tả vai trò (tư vấn xe, so sánh xe, phân tích thống kê, v.v.) và quy tắc trả lời.
- **Knowledge Context:** Ngữ cảnh đã truy xuất (SQL matches + semantic matches + features + images).
- **Customer Profile (JSON):** Hồ sơ người dùng hiện tại để cá nhân hóa câu trả lời.
- **Lịch sử hội thoại (chat_history):** Tối đa 10 lượt gần nhất, duy trì ngữ cảnh phiên trò chuyện.
- **Câu hỏi hiện tại:** Nội dung người dùng yêu cầu.

Một số quy tắc chung trong system prompt: (1) hiển thị giá với dấu phẩy phân tách (ví dụ: \$21,950); (2) **không** thêm link "View Details" vào nội dung câu trả lời — ứng dụng tự hiển thị vehicle cards bên dưới; (3) trả lời bằng ngôn ngữ của người dùng; (4) không đề xuất brand nằm trong `excluded_brands` của hồ sơ.

4.5.4 Chuẩn hóa câu hỏi (Standalone Question)

Trước khi truy xuất, câu hỏi hiện tại được chuẩn hóa thành **standalone question**: nếu câu hỏi phụ thuộc ngữ cảnh trước đó (câu hỏi nối tiếp), LLM chuyển đổi thành câu hỏi độc lập chứa đủ thông tin. Câu hỏi chuẩn hóa này được dùng để tạo embedding và tìm kiếm Qdrant, đảm bảo kết quả retrieval chính xác.

4.6 Hạn chế và định hướng phát triển

4.6.1 Hạn chế hiện tại

- **UserProfile lưu in-memory:** Hồ sơ người dùng (`UserProfile`) được lưu in-memory per `session_id` trong tiến trình backend. Khi backend restart hoặc Cloud Run scale down instance, toàn bộ hồ sơ mất. Đây là thiết kế có chủ đích cho Cloud Run (`-max-instances=1`) nhưng không phù hợp khi scale ngang.
- **Không có persistence cho chat sessions:** Bảng `gold.chat_sessions` và `gold.chat_messages` đã được thiết kế trong schema nhưng chatbot hiện tại lưu lịch sử hội thoại in-memory (mảng `chat_history_buffer`), không ghi vào database.
- **Ngưỡng score Qdrant cố định:** Score threshold để lọc kết quả Qdrant (`score > 1.3`) là giá trị cố định, chưa được tối ưu tự động theo độ phân bố dữ liệu.

4.6.2 Định hướng phát triển

- **Persistent chat sessions:** Ghi lịch sử hội thoại và `UserProfile` vào database (`gold.chat_sessions`, `gold.chat_messages`) để hỗ trợ multi-instance deployment và khôi phục phiên.
- **Tích hợp với Recommendation Engine:** Khi chatbot gợi ý xe, ghi lại tương tác (`gold.user_interactions`)

để recommendation engine học từ hành vi qua chatbot.

- **Tối ưu hóa retrieval:** Thử nghiệm các chiến lược retrieval nâng cao như re-ranking (cross-encoder) hay Hypothetical Document Embeddings (HyDE) để cải thiện chất lượng kết quả ngữ nghĩa.
- **Đánh giá tự động:** Xây dựng bộ test cases (câu hỏi + expected answer) để đánh giá định lượng chất lượng câu trả lời chatbot sau mỗi lần cập nhật dữ liệu.

5 TỔNG KẾT

5.1 Kết Luận

Car Recommendation System là một hệ thống tích hợp hoàn chỉnh, kết hợp **Data Engineering, Machine Learning, Natural Language Processing** và **modern web technologies** để giải quyết bài toán tìm kiếm và tư vấn mua xe thông minh.

Những đóng góp kỹ thuật nổi bật của dự án:

- **Pipeline dữ liệu tự động hoàn toàn:** Triển khai pipeline thu thập và xử lý dữ liệu hàng tuần sử dụng **Temporal** (orchestration) với kiến trúc fail-stop 3-workflow: WeeklyCrawl → Transform → ML. Dữ liệu trải qua **Medallion Architecture** (Bronze → Silver → Gold) được quản lý bởi **dbt**, đảm bảo tính incremental và idempotent.
- **Hệ thống đề xuất 3-stage hybrid:** Thiết kế Recommendation Engine theo pipeline **Candidate Generation → Ranking → Re-ranking**. Stage 1 gồm 4 recaller song song (Collaborative, Content, Vector, Popularity); Stage 2 dùng **WeightedLinearRanker** với 7 features có trọng số cấu hình qua YAML; Stage 3 dùng **MMRReranker** (Maximal Marginal Relevance) để đảm bảo đa dạng kết quả với hard caps theo brand/model.
- **Chatbot tư vấn Agentic RAG:** Xây dựng chatbot trên **LangGraph StateGraph** với 7 loại intent routing (compare, analytics, specs, specific, vague, chitchat, off_topic), cơ chế **slot-fill profile** và **hybrid retrieval** (SQL hard-filter + Qdrant semantic search trên collection **car_vectorize**).
- **Kiến trúc database Medallion:** Thiết kế multi-schema (Bronze/Silver/Gold) với Silver theo chuẩn 3NF (dimensional model) do dbt quản lý, và Gold gồm denormalized marts phục vụ frontend cùng bảng user-domain do ứng dụng ghi trực tiếp.
- **Triển khai sản xuất trên GCP:** Hệ thống được containerized và triển khai lên **Google Cloud Platform** với Cloud Run (backend + frontend), AlloyDB PostgreSQL 17, Qdrant Cloud, và Temporal Cloud — truy cập tại carsalesfinder.com.

5.2 Ứng Dụng Thực Tế

Hệ thống có thể được triển khai trong các scenarios thực tế sau:

1. **E-commerce platforms** cho showrooms và đại lý xe: cung cấp trải nghiệm mua sắm cá nhân hóa với AI chatbot tư vấn 24/7, hỗ trợ so sánh xe và gợi ý thông minh theo hành vi người dùng.
2. **Automotive marketplaces** (như các nền tảng mua bán xe trực tuyến): giúp buyers tìm xe phù hợp nhanh chóng qua natural language search thay vì form-based filtering phức tạp, đồng thời phân tích thống kê inventory theo thời gian thực.
3. **Customer analytics platform** cho manufacturers và dealers: thu thập insights từ user behavior data (lượt xem, click, lưu yêu thích, liên hệ) để tối ưu inventory và chiến lược định giá.
4. **Nền tảng có thể mở rộng:** Kiến trúc pipeline Temporal + dbt + vector database có thể adapt cho các vertical markets khác như bất động sản, điện tử tiêu dùng, hoặc bất kỳ domain nào cần personalized search và recommendations trên dữ liệu crawled định kỳ.